LABORATORY RECORD

Course: Full Stack Web Development

Course Code: 23CM4121

Experiment No.: 3

Student Name: K.Vinay

Roll No.: A24126552086

Date: 31-08-2026

---

1. TITLE

Implementation of Array Operations and Functions in JavaScript

2. AIM

To demonstrate the use of fundamental JavaScript array manipulation methods and custom functions to process numeric and string data.

3. OBJECTIVE

The objectives of this experiment are:

- To declare and manipulate arrays in JavaScript.
  \n
- To use array methods like push, pop, shift, and unshift.
  \n
- To write custom functions for iterating over array elements.
  \n
- To calculate the sum of numerical array elements dynamically.

4. THEORY

JavaScript arrays are versatile data structures that hold collections of items. Built-in methods like push() and pop() modify the array at the end, while unshift() and shift() modify it at the beginning. Functions encapsulate logic and can process arrays iteratively using loops.

5. ALGORITHM / PROCEDURE

1. Declare a string array named 'fruits' with initial values.
   \n
2. Output the array length and access the first element.
   \n
3. Add and remove elements using push, pop, shift, and unshift methods.
   \n
4. Sort the array using the sort() method.
   \n
5. Create a function 'displayArray' that prints all elements using a loop.
   \n
6. Create a function 'findSum' that returns the sum of elements in a numeric array.

\n
7. Output the sum to the console.

6. PROGRAM

arraysandfns.js

```
let fruits = ["Apple", "Banana", "Mango", "Orange"];

console.log("Original Array:", fruits);

// Array property
console.log("Array Length:", fruits.length);

// Access element
console.log("First Element:", fruits[0]);

// Add element at end
fruits.push("Grapes");
console.log("After push:", fruits);

// Remove element from end
fruits.pop();
console.log("After pop:", fruits);

// Add element at beginning
fruits.unshift("Pineapple");
console.log("After unshift:", fruits);

// Remove element from beginning
fruits.shift();
console.log("After shift:", fruits);

// Sort array
fruits.sort();
console.log("After sort:", fruits);


// Function to display array
function displayArray(arr) {
    console.log("Array Elements:");

    for (let i = 0; i < arr.length; i++) {
        console.log(arr[i]);
    }
}

// Calling function
displayArray(fruits);


// Function to find sum
function findSum(numbers) {
    let sum = 0;

    for (let i = 0; i < numbers.length; i++) {
```

```
        sum = sum + numbers[i];
    }

    return sum;
}

// Number array
let numbers = [10, 20, 30, 40, 50];

console.log("Numbers:", numbers);
console.log("Sum:", findSum(numbers));
```

7. CODE EXPLANATION

Code / Syntax	Explanation
fruits.push('Grapes')	Appends a new item to the end of the array.
fruits.shift()	Removes and returns the first element of the array.
function displayArray(arr)	Defines a reusable block of code to loop over elements.
console.log(...)	Prints standard output and variables to the terminal.

8. TEST CASES AND EXECUTION DATA

Test Case	Input / Action	Expected Result	Actual Result	Status
TC-01	Array Manipulation	Execute push/pop	Array updates accordingly	**Pass**
TC-02	Function Sum	Call findSum()	Correct mathematical sum is returned	**Pass**

9. OUTPUT

```
Original Array: [ 'Apple', 'Banana', 'Mango', 'Orange' ]
Array Length: 4
First Element: Apple
After push: [ 'Apple', 'Banana', 'Mango', 'Orange', 'Grapes' ]
After pop: [ 'Apple', 'Banana', 'Mango', 'Orange' ]
After unshift: [ 'Pineapple', 'Apple', 'Banana', 'Mango', 'Orange' ]
After shift: [ 'Apple', 'Banana', 'Mango', 'Orange' ]
After sort: [ 'Apple', 'Banana', 'Mango', 'Orange' ]
Array Elements:
Apple
Banana
Mango
Orange
```

```
Numbers: [ 10, 20, 30, 40, 50 ]
Sum: 150
```

10. RESULT

Thus, array manipulations and function executions in JavaScript were successfully demonstrated and validated via standard output.